

Kubernetes Ingress: The Ultimate Beginner's Guide

Kubernetes Ingress acts as the smart receptionist for your cluster, directing external web traffic to the correct internal services. Understanding how to configure and debug it is one of the most critical skills you will use daily in cloud operations.

What you'll learn:

- 1 Setting Up Your First Ingress Controller
- 2 Routing Traffic by URL Path
- 3 Securing Your Apps with HTTPS
- 4 Troubleshooting 502 and 404 Errors

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Setting Up Your First Ingress Controller

Before you can route traffic, you need an Ingress Controller active in your cluster. Think of the Controller as the actual engine (like NGINX or Traefik) that executes your routing rules. Without it, your Ingress configuration files will just sit there and do nothing. You can easily spin one up in your local test environment with a single command.

```
$minikube addons enable ingress
```

Cloud & DevOps Daily

Kubernetes Ingress: Route Traffic to the Right Service

Monday, September 21, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Routing Traffic by URL Path

Path-based routing lets you host multiple microservices behind a single domain name. For example, requests to 'example.com/api' go to your backend service, while 'example.com/' goes to your frontend. We can generate a quick template using kubectl's dry-run feature to see exactly how this configuration looks before applying it.

```
$kubectl create ingress path-ingress --rule="example.com/api*=backend-svc:80" --dry-run=client -o yaml
```

Cloud & DevOps Daily

Kubernetes Ingress: Route Traffic to the Right Service

Monday, September 21, 2026

Swipe to tip 3 >> linkedin.com/in/mrmanrajsingh

3

TIP 3 OF 4

Securing Your Apps with HTTPS

To secure your web application, you must configure TLS termination at the Ingress level. This means the Ingress handles the encryption/decryption, saving your backend pods from doing heavy cryptographic work. First, create a Kubernetes secret containing your SSL certificate and private key, then reference it in your Ingress file.

```
$kubectl create secret tls app-tls-cert --cert=tls.crt --key=tls.key
```

Cloud & DevOps Daily

Kubernetes Ingress: Route Traffic to the Right Service

Monday, September 21, 2026

Swipe to tip 4 >> linkedin.com/in/mrmanrajsingh

4

TIP 4 OF 4

Troubleshooting 502 and 404 Errors

When traffic doesn't reach your app, the Ingress Controller logs are your best friend. A 502 Bad Gateway usually means the Ingress exists, but your backend Service or Pod is down. A 404 Not Found means your Ingress rules don't match the URL path you typed. Use this command to stream live traffic logs and pinpoint the error.

```
$kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx --tail=50 --follow
```

Cloud & DevOps Daily

Kubernetes Ingress: Route Traffic to the Right Service

Monday, September 21, 2026

See Key Takeaways on next page >> linkedin.com/in/mrmanrajsingh

Key Takeaways

- + An Ingress resource defines the routing rules, but the Ingress Controller actually enforces them.
- + Always verify your target Services are running on the exact port specified in your Ingress rules.
- + Use 'kubectl get ingress' to find the external IP address assigned to your cluster.
- + Path-based routing is case-sensitive and depends on the order of rules in your YAML file.
- + Minikube users need to run 'minikube tunnel' to assign an IP address to their local Ingress.

Watch Out For

- ! Forgetting to install an Ingress Controller, expecting the Ingress YAML to work out of the box.
- ! Mismatched service ports: pointing the Ingress to port 80 when the Service is listening on port 8080.
- ! Creating the Ingress resource in a different namespace than the target backend Service.
- ! Not updating your local /etc/hosts file when testing custom domain names on your local machine.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh